

Course Prerequisite Planner

Struktur Data dan Pemrograman Berorientasi Objek

Daftar Anggota Kelompok 4:

- 5027251030 - Muhammad Syadzili Abdul Muhyi
- 5027251060 - Husam Danish
- 5027251090 - Rheza Pramudita Adi Putra
- 5027251102 - Achmad Rifqy Aqila
- 5027251108 - Nadia Iqlima Al Fairuz

1. Daftar Isi

Course Prerequisite Planner.....	1
1. Daftar Isi.....	1
2. Pendahuluan.....	1
3. Rumusan Masalah & Batasan.....	2
3.1. Rumusan Masalah.....	2
3.2. Batasan.....	2
4. Dataset.....	2
5. Perancangan Sistem (Design Decision Log).....	3
5.1. Struktur Data.....	3
5.2. Algoritma Graph.....	4
5.3. Diagram Kelas (Ringkas).....	6
6. Implementasi.....	6
7. Pengujian.....	7
7.1. Skenario Normal.....	7
7.2. Skenario Edge Case.....	8
7.2.1 Skenario Edge Case 1: Pencarian Gagal (Penanganan Null Pointer).....	8
7.2.2 Skenario Edge Case 2: Cycle Detection (Konflik Kurikulum).....	8
7.2.3 Skenario Edge Case 3: Menambah Data Duplikat.....	10
7.3. Tracing Manual (Pohon, Graf, dan Edge Case).....	10
7.3.1. Tracing Proses Tree: Penyisipan dan Rotasi AVL.....	10
7.3.2. Tracing Proses Graph: Topological Sort (Skenario Normal).....	11
7.3.3. Tracing Edge Case: Deteksi Siklus (Konflik Kurikulum).....	11
8. Analisis Kompleksitas.....	12
9. What-If Analysis.....	12
10. Hasil & Demonstrasi.....	13
10.1. Cari matkul berdasarkan kode / prefix.....	13

10.2. Tampilkan prasyarat langsung & tidak langsung.....	14
10.3. Rekomendasi urutan pengambilan (Topological Sort).....	16
10.4. Deteksi siklus prasyarat (Cycle Detection).....	17
10.5. Telusuri rantai matkul dari suatu titik (DFS).....	18
10.6. Tampilkan semua matkul (In-Order AVL).....	19
10.7. Tampilkan seluruh graph prasyarat.....	20
10.8. SIMULASI: Tambah matkul baru & lihat dampaknya.....	21
10.9. Tambah matkul baru.....	22
10.10. Hapus matkul.....	23

2. Pendahuluan

Dalam pelaksanaan kurikulum di tingkat perguruan tinggi, kelancaran studi mahasiswa sangat bergantung pada kemampuan mereka dalam merencanakan pengambilan mata kuliah. Struktur kurikulum yang berjenjang mensyaratkan kelulusan pada mata kuliah dasar sebelum dapat mengambil mata kuliah lanjutan. Kesalahan dalam menyusun urutan ini seringkali mengakibatkan keterlambatan penyelesaian studi, bentrokan jadwal, hingga ketidakseimbangan beban akademik per semester.

Oleh karena itu, sistem yang mampu memetakan rantai prasyarat (prerequisites) secara terstruktur menjadi sangat penting. Aplikasi "Course Prerequisite Planner" dirancang untuk menjawab tantangan tersebut. Melalui pendekatan berbasis struktur data graf dan pohon, aplikasi ini bertujuan untuk mengotomatisasi penyusunan jalur pembelajaran, memastikan urutan pengambilan valid secara logika, serta mencegah adanya kebuntuan kurikulum atau siklus yang tidak logis.

3. Rumusan Masalah & Batasan

3.1. Rumusan Masalah

- Bagaimana merancang sebuah sistem yang secara otomatis mampu menemukan urutan pengambilan mata kuliah yang valid berdasarkan aturan prasyarat kurikulum?
- Bagaimana cara mendeteksi konflik atau siklus tidak valid (circular dependencies) di dalam aturan kurikulum yang menghalangi mahasiswa menyelesaikan studinya?

3.2. Batasan

- Implementasi teknis dibatasi pada pemanfaatan tepat 1 jenis Tree (AVL Tree) dan 1 struktur Graph (Directed Graph).
- Aplikasi harus menjalankan setidaknya dua algoritma Graph yang berbeda (contoh: Kahn's Algorithm untuk Topological Sort dan DFS untuk Cycle Detection).
- Dataset yang digunakan bersifat lokal dan dikonstruksi secara mandiri (buatan sendiri), tidak

terhubung ke API SIAKAD eksternal.

4. Dataset

Aplikasi ini mendayagunakan data struktural yang dibentuk dalam format Comma-Separated Values (CSV). Pembuatan data dikoordinasikan secara manual oleh Anggota 1 untuk menyimulasikan data perkuliahan secara realistis.

- **courses.csv**: Merupakan basis data utama untuk detail mata kuliah. Berisi atribut kritis berupa Kode (Primary Key), Nama Mata Kuliah, Bobot SKS, Semester yang direkomendasikan, Tipe Kelas (Wajib atau Pilihan), Kuota maksimal tiap kelas, dan Departemen penyelenggara mata kuliah.
- **prerequisites.csv**: Merepresentasikan himpunan garis (edges) pada graf. File ini mendefinisikan relasi ketergantungan dari kolom pertama "Lanjutan" (Kode mata kuliah yang akan diambil) dan kolom kedua "Syarat" (Kode mata kuliah prasyaratnya).

5. Perancangan Sistem (Design Decision Log)

5.1. Struktur Data

	Keputusan	Pilihan yang Dipilih	Alternatif yang Ditolak	Alasan
1	Struktur data penyimpanan & pencarian matkul	AVL Tree	Hashmap / array biasa	AVL Tree menjamin pencarian $O(\log n)$ dan hasil selalu terurut secara alfabetis (in-order traversal), berguna untuk fitur "tampilkan semua matkul". HashMap lebih cepat $O(1)$ tapi tidak terurut dan tidak mendukung prefix search
2	Representasi relasi prasyarat antar matkul	Directed Graph dengan Adjacency List	Adjacency Matrix / Tabel 2D	Adjacency List lebih hemat memori untuk graph jarang (sparse), karena sebagian besar matkul hanya punya 1-3 prasyarat. Matrix akan membuat tabel 27×27 yang sebagian

				besar kosong
3	Algoritma urutan pengambilan matkul	Topological Sort — Kahn's Algorithm	DFS-based Topological Sort	Kahn's Algorithm secara alami mendeteksi siklus (jika <code>result.size() != V</code> , ada cycle). Lebih mudah diintegrasikan dengan PriorityQueue sehingga output deterministik dan mudah dibaca
4	Deteksi prasyarat melingkar	DFS 3-warna (WHITE/GRAY/BLACK)	DFS sederhana / Union-Find	3-warna membedakan node yang <i>sedang diproses</i> (GRAY) vs <i>sudah selesai</i> (BLACK), sehingga path siklus bisa dilacak dan ditampilkan ke pengguna. Union-Find tidak bisa menampilkan path siklusnya
5	Fitur simulasi matkul baru	Tambah node+edge ke graph yang sudah ada, lalu re-run topological sort	Buat graph baru dari awal setiap simulasi	Lebih efisien karena hanya menambahkan node dan edge baru tanpa rebuild seluruh graph $O(V+E)$. Memungkinkan simulasi berulang dalam satu sesi
6	Format penyimpanan data matkul	CSV (courses.csv + prerequisites.csv)	Hardcode di source code / database	CSV mudah diedit tanpa mengubah kode, portabel (tidak butuh install database), dan cukup untuk skala dataset kuliah satu jurusan
7	Arsitektur program	Package terpisah: model, tree, graph, utils	Semua dalam satu file/class	Setiap anggota bisa mengerjakan package-nya masing-masing tanpa konflik. System Integrator cukup meng-import package yang sudah jadi. Mencerminkan prinsip

				<i>separation of concerns</i>
--	--	--	--	-------------------------------

5.2. Algoritma Graph

Pada sistem Course Prerequisite Planner, graph digunakan untuk merepresentasikan hubungan prasyarat antar mata kuliah. Setiap mata kuliah direpresentasikan sebagai vertex (simpul), sedangkan hubungan prasyarat direpresentasikan sebagai directed edge (sisi berarah). Sebagai contoh, hubungan IT101 → IT202 menunjukkan bahwa IT101 harus diselesaikan sebelum mahasiswa dapat mengambil IT202.

Graph diimplementasikan menggunakan representasi adjacency list berbasis `HashMap<String, List<String>>` karena lebih efisien dalam penggunaan memori dan memudahkan proses traversal. Untuk menelusuri hubungan antar mata kuliah digunakan algoritma Depth-First Search (DFS), yang bekerja dengan mengunjungi suatu vertex kemudian menelusuri seluruh vertex yang terhubung secara mendalam sebelum kembali ke vertex sebelumnya (*backtracking*).

Implementasi DFS dilakukan secara rekursif dengan memanfaatkan struktur data `visited` untuk memastikan setiap vertex hanya dikunjungi satu kali. Pendekatan ini memungkinkan proses penelusuran hubungan prasyarat berjalan secara efisien dan terhindar dari pengulangan yang tidak diperlukan.

```
public void dfs(String startCourse) {
    Set<String> visited = new HashSet<>();
    dfsHelper(startCourse, visited);
}

private void dfsHelper(String course, Set<String> visited) {
    visited.add(course);
    System.out.println(course);

    for (String neighbor :
        adjacencyList.getOrDefault(course,
            new ArrayList<>())) {

        if (!visited.contains(neighbor)) {
            dfsHelper(neighbor, visited);
        }
    }
}
```

Selain proses traversal, graph juga dibangun secara otomatis menggunakan data yang berasal dari file `prerequisites.csv`. Setiap baris pada file tersebut berisi pasangan mata kuliah prasyarat dan mata kuliah tujuan yang kemudian dikonversi menjadi edge pada graph. Dengan pendekatan ini, perubahan dataset dapat dilakukan tanpa perlu mengubah struktur program secara langsung.

Secara teoritis, algoritma DFS memiliki kompleksitas waktu sebesar $O(V + E)$, dengan V menyatakan jumlah vertex dan E menyatakan jumlah edge pada graph. Kompleksitas tersebut diperoleh karena setiap vertex dan setiap edge hanya dikunjungi maksimal satu kali selama proses traversal berlangsung. Oleh karena itu, DFS merupakan pilihan yang sesuai untuk digunakan pada sistem ini mengingat jumlah data mata kuliah dan hubungan prasyarat yang terus bertambah seiring perkembangan kurikulum.

Topological Sort Merupakan kunci untuk menyelesaikan masalah urutan valid. Implementasi menggunakan Kahn's Algorithm, yaitu pendekatan berbasis BFS yang bekerja dengan menghitung *in-degree* (jumlah prasyarat yang belum terpenuhi) untuk setiap mata kuliah. Mata kuliah dengan *in-degree* 0 (tanpa prasyarat) dimasukkan ke dalam queue terlebih dahulu, kemudian diproses satu per satu sambil mengurangi *in-degree* mata kuliah yang bergantung padanya. Algoritma ini menjamin setiap mata kuliah didaftarkan hanya setelah seluruh prasyaratnya dievaluasi terlebih dahulu.

Cycle Detection Memanfaatkan DFS dengan penandaan tiga warna (WHITE, GRAY, BLACK) untuk memastikan tidak ada prasyarat yang berputar (misal A mensyaratkan B, dan B mensyaratkan A). Setiap node yang belum dikunjungi berstatus WHITE, node yang sedang berada dalam jalur DFS aktif berstatus GRAY, dan node yang telah selesai dijelajahi berstatus BLACK. Apabila proses DFS menemukan node bertetangga yang masih berstatus GRAY, hal tersebut menandakan adanya *back-edge* yang mengarah ke leluhur node tersebut dalam jalur penjelajahan saat ini. Kondisi inilah yang disebut siklus.

5.3. Diagram Kelas (Ringkas)

Struktur relasional program dibagi ke dalam beberapa kelas utama, yaitu Course (model data), AVLNode dan AVLTree (manajemen tree), CourseGraph (manajemen graf dan algoritma), dan kelas pengendali aplikasi yaitu Main.

6. Implementasi

Berikut adalah pemetaan fail *source code* terhadap fungsionalitasnya (berada di dalam direktori `final_project_strukdat/`):

- `src/model/Course.java`: Merupakan representasi entitas matkul yang merangkum properti pendukung dari dataset *courses*.
- `src/tree/AVLTree.java`: Menangani struktur dan keseimbangan hierarki data dengan mekanisme *rotation* (LL, RR, LR, RL) di setiap pasca-penyisipan *node*.
- `src/graph/CourseGraph.java`: Mengimplementasikan struktur data *Directed Graph*

menggunakan representasi *Adjacency List* berbasis `HashMap<String, List<String>>`. Kelas ini bertanggung jawab membangun relasi prasyarat antar mata kuliah dari dataset `prerequisites.csv`, menampilkan struktur graph, melakukan validasi keberadaan mata kuliah, serta menjalankan algoritma *Depth-First Search (DFS)* untuk menelusuri hubungan prasyarat dari suatu mata kuliah ke mata kuliah lain yang bergantung padanya.

- `src/Main.java`: Modul antarmuka *Command-Line Interface (CLI)* yang memproses interaksi pengguna berdasarkan *flowchart* aplikasi.

```
// Pseudocode Flowchart Interaksi CLI
START
LOAD CSV Data into AVL Tree and CourseGraph
WHILE (True):
    PRINT Menu Utama:
        [TREE - AVL Tree]
        1. Cari matkul berdasarkan kode / prefix
        9. Tambah matkul baru
        10. Hapus matkul
        [GRAPH - Directed Graph]
        2. Tampilkan prasyarat langsung & tidak langsung
        3. Rekomendasi urutan pengambilan (Topological Sort)
        4. Deteksi siklus prasyarat (Cycle Detection)
        5. Telusuri rantai matkul dari suatu titik (DFS)
        7. Tampilkan seluruh graph prasyarat
        [UMUM]
        6. Tampilkan semua matkul (In-Order AVL)
        8. SIMULASI: Tambah matkul baru & lihat dampaknya
        0. Keluar
    INPUT choice
    SWITCH (choice):
        CASE 1: Search Course in AVL Tree -> PRINT Detail
        CASE 2: DFS Traversal on CourseGraph -> PRINT Path
        CASE 3: Run Topological Sort -> PRINT Sequence
        CASE 4: Run Cycle Detection -> PRINT Status
        CASE 5: Run DFS dari node tertentu -> PRINT Path
        CASE 6: In-Order Traversal AVL Tree -> PRINT Semua Matkul
```

```
CASE 7: Display Graph Adjacency List -> PRINT Graph
CASE 8: Insert Temporary Node & Edges -> Run Topo Sort ->
PRINT Dampak
CASE 9: Insert Node to AVL & Graph -> Add Edges
CASE 10: Delete Node dari AVL & Graph
CASE 0: EXIT

END
```

7. Pengujian

7.1. Skenario Normal

- **Skenario 1 (Pencarian):** Mencari mata kuliah dengan kode yang valid. Diharapkan sistem mengembalikan rincian data dalam kompleksitas waktu yang sangat minimal.
- **Skenario 2 (Urutan):** Mencari urutan pengambilan matkul. Diharapkan hasil *Topological Sort* mengeluarkan urutan mulai dari mata kuliah dasar ke lanjutan.
- **Skenario 3 (Simulasi Tambah Data):** Menambah mata kuliah baru dan prasyaratnya secara *runtime*. Diharapkan graf diperbarui tanpa mengganggu relasi awal.

7.2. Skenario Edge Case

- **Skenario 1 (Pencarian Gagal):** Mencari nilai kode unik yang fiktif atau tidak terdaftar, untuk menguji kehandalan penanganan *null pointer*. Diharapkan program tidak crash, tapi menampilkan pesan 'Mata kuliah tidak ditemukan'.
- **Skenario 2 (Deteksi Siklus):** Menyuntikkan relasi yang berlawanan pada *graph* untuk memicu loop tak berhingga, lalu melihat apakah *Cycle Detection* memutuskan eksekusi dan mengeluarkan pesan error.
- **Skenario 3 (Data Duplikat):** Mengimpor kode matkul yang sama dua kali. AVL Tree diharapkan menolak duplikasi demi menjaga integritas data.

7.2.1 Skenario Edge Case 1: Pencarian Gagal (Penanganan Null Pointer)

Untuk menguji kehandalan sistem saat menerima input yang tidak valid, dilakukan simulasi pencarian mata kuliah dengan kode fiktif (misalnya [ZXC999](#)).

Tracing proses pencarian di AVL Tree:

1. Input [ZXC999](#) diterima oleh sistem.
2. AVL Tree memulai pencarian dari *root*. Karena [ZXC999](#) tidak ada di dalam tree, traversal akan terus bergerak ke arah *leaf* (daun) mengikuti aturan perbandingan string (alfabetis).

3. Saat pencarian mencapai ujung *tree* dan mengevaluasi *child* yang kosong, pencarian mengembalikan nilai `null` atau `empty list`.

Output program:

```
--- CARI MATKUL ---  
Masukkan kode / prefix (contoh: IT3, IT401): ZXC999  
Tidak ada matkul dengan prefix 'ZXC999'.
```

Sistem berhasil menangkap nilai kosong pada modul antarmuka pengguna tanpa memicu `NullPointerException` atau menghentikan eksekusi program (*crash*). Program secara mulus mengembalikan kontrol ke pengguna untuk kembali ke Menu Utama.

7.2.2 Skenario Edge Case 2: Cycle Detection (Konflik Kurikulum)

Karena logika antarmuka CLI dirancang sangat kuat untuk menolak input prasyarat yang tidak valid secara interaktif, maka pengujian ini disimulasikan dengan **mengedit file `prerequisites.csv` secara manual**. Untuk memicu konflik kurikulum (misalnya mata kuliah A mensyaratkan B, namun B juga mensyaratkan A), ditambahkan satu baris relasi prasyarat buatan `IT101 - IT202` ke bagian bawah file CSV. Padahal, relasi `IT202 - IT101` (`IT101 - IT202`) sudah ada di dataset asli. Setelah program dijalankan ulang dan Menu 4 dipilih, hal ini sukses menciptakan siklus dua arah yang dapat dideteksi oleh sistem: `IT101 - IT202 - IT101`.

(Catatan: Detail tracing proses DFS dari simpul ke simpul hingga mendeteksi siklus dapat dilihat pada bagian 7.1 Tracing Manual di bawah).

Output program:

```
Pilihan Anda: 4  
  
===== CYCLE DETECTION =====  
Siklus ditemukan: IT202 -> IT101 -> IT202  
Kurikulum TIDAK VALID! Ada prasyarat melingkar.  
  
-----
```

Sebagai konsekuensi, Topological Sort juga tidak dapat menyelesaikan urutan secara lengkap, karena in-degree IT101 dan IT202 tidak akan pernah mencapai 0 (keduanya saling menunggu satu sama lain):

```
Pilihan Anda: 3
```

```
===== TOPOLOGICAL SORT (Urutan Pengambilan MK) =====  
PERINGATAN: Topological sort tidak lengkap!  
Kemungkinan ada SIKLUS pada graph prasyarat.  
Berhasil diurutkan: 8 dari 29 matkul.  
-----
```

Penanganan Konflik Kurikulum

Berdasarkan hasil tracing di atas, sistem menangani konflik kurikulum dengan mekanisme berikut: ketika DFS menemukan *back-edge* menuju node berwarna GRAY, sistem mencatat seluruh jalur node dari titik tersebut hingga node GRAY yang ditemukan sebagai bagian dari siklus, lalu menandai kurikulum sebagai **tidak valid**. Validitas ini juga tercermin dari hasil Topological Sort yang gagal mengurutkan seluruh node (jumlah node terurut < jumlah total node), karena node-node dalam siklus memiliki in-degree yang tidak pernah mencapai nol.

Solusi yang direkomendasikan sistem terhadap konflik semacam ini ada tiga: (1) menghapus salah satu relasi prasyarat dalam siklus sehingga hierarki menjadi satu arah, (2) mengubah relasi tersebut menjadi *co-requisite* (mata kuliah yang dapat diambil secara bersamaan dalam semester yang sama, bukan berurutan), atau (3) memecah salah satu mata kuliah menjadi dua bagian terpisah (misalnya teori dan praktikum) agar ketergantungan dapat diselesaikan secara bertahap.

7.2.3 Skenario Edge Case 3: Menambah Data Duplikat

Demi mengamankan integritas struktur data graf dan pohon, aplikasi harus kebal terhadap anomali duplikasi. Disimulasikan pengguna mencoba mengimpor atau menambahkan mata kuliah yang kodenya sudah bersemayam di memori (contoh: mencoba menambahkan **IT101** dua kali lewat menu "Tambah Mata Kuliah").

Tracing proses penolakan data:

1. Input pembuatan mata kuliah **IT101** direkam pengguna.
2. Sebelum *instantiation* kelas **Course** dan manipulasi *pointer* di AVL Tree, metode pengontrol di **Main.java** secara proaktif memanggil fungsi validasi **graph.hasCourse("IT101")**.
3. Pengecekan secara instan (kompleksitas $O(1)$ pada HashMap graf) mengevaluasi eksistensi kunci tersebut dan mengembalikan boolean **True**.

Output program:

```
--- TAMBAH MATA KULIAH ---  
Kode (contoh: IT601): IT101  
[!] Matkul 'IT101' sudah ada.
```

Proses *insert* dibatalkan seketika di gerbang validasi logika CLI. Langkah preventif ini memastikan bahwa AVL Tree tidak mengalami kerusakan struktur akibat duplikasi properti *Primary Key*, serta Adjacency List di Graph tetap murni tanpa ada tumpukan *node* kembar.

7.3. Tracing Manual (Pohon, Graf, dan Edge Case)

Sesuai dengan ketentuan kewajiban pelaporan proyek, berikut adalah penjabaran proses di balik layar (*tracing manual*) untuk tiga komponen utama sistem:

7.3.1. Tracing Proses Tree: Penyisipan dan Rotasi AVL

Mensimulasikan masuknya 3 mata kuliah baru secara berurutan: **IT300**, **IT200**, **IT100** ke dalam AVL Tree yang masih kosong untuk melihat bagaimana *tree* menyeimbangkan dirinya.

- **Langkah 1:** Insert **IT300**. Menjadi *root*. Tinggi = 1, Balance Factor = 0.
- **Langkah 2:** Insert **IT200**. Karena **IT200** < **IT300**, diletakkan di anak kiri. Tinggi root = 2, Balance Factor root = 1.
- **Langkah 3:** Insert **IT100**. Karena **IT100** < **IT200**, diletakkan di anak kiri **IT200**. Pohon menjadi condong ke kiri (*Left-Left Heavy*).
- **Deteksi Ketidakseimbangan:** Balance Factor dari *root* (**IT300**) menjadi 2. Hal ini memicu fungsi *rebalancing* otomatis.
- **Aksi Rotasi:** Karena kondisinya *Left-Left*, dilakukan **Right Rotation** tunggal pada *node* **IT300**.
- **Hasil Akhir:** **IT200** naik menjadi *root* baru, **IT100** tetap menjadi anak kiri **IT200**, dan **IT300** turun menjadi anak kanan **IT200**. Pohon kembali seimbang dengan tinggi 2 dan Balance Factor 0.

7.3.2. Tracing Proses Graph: Topological Sort (Skenario Normal)

Menggunakan dataset **courses.csv** dan **prerequisites.csv** yang berisi 27 mata kuliah dan 41 relasi prasyarat, algoritma Kahn (Berdasarkan BFS) berjalan sebagai berikut:

1. Menghitung *in-degree* (jumlah prasyarat masuk) seluruh node. Mata kuliah semester 1 (IT101–IT105) memiliki *in-degree* 0.
2. Kelima mata kuliah tersebut dimasukkan ke dalam antrean (*queue*) dan diproses pertama, menghasilkan urutan keluaran awal: IT101, IT102, IT103, IT104, IT105.
3. Setelah IT101 diproses dan dikeluarkan dari antrean, *in-degree* dari mata kuliah lanjutannya (seperti IT202, IT204) dikurangi 1. Begitu *in-degree* salah satu mata kuliah tersebut mencapai 0, ia akan didorong ke dalam *queue*.
4. Proses berulang terus hingga seluruh antrean kosong.
5. **Output:** Menghasilkan 27 mata kuliah yang terurut sempurna. Karena jumlah *node* hasil sama dengan jumlah total *node*, kurikulum dinyatakan **valid**.

7.3.3. Tracing Edge Case: Deteksi Siklus (Konflik Kurikulum)

Mensimulasikan keadaan setelah file `prerequisites.csv` disabotase secara manual untuk menyuntikkan relasi melingkar: **IT101 - IT202 - IT101**. Modifikasi DFS mendeteksi hal ini dengan bantuan penandaan tiga warna (WHITE, GRAY, BLACK).

Langkah	Node Kunjungan	Warna Sebelum	Aksi Algoritma	Warna Sesudah
1	IT101	WHITE	DFS dimulai dari IT101, ditandai sedang aktif dieksplorasi di call stack saat ini.	GRAY
2	IT202	WHITE	Melangkah ke tetangga dari IT101 (lewat edge IT101→IT202). Lanjut DFS ke dalam.	GRAY
3	IT101	GRAY	Tetangga dari IT202 (lewat edge IT202→IT101) adalah IT101 yang masih berstatus GRAY.	tetap GRAY
Hasil	-	-	Karena menabrak node GRAY, Back-edge ditemukan! Siklus terkonfirmasi. DFS langsung menghentikan proses.	-

Dampak dari siklus ini adalah Topological Sort juga akan berhenti di tengah jalan (hanya mengurutkan sebagian kecil matkul) karena sisa mata kuliah yang terjebak di siklus tidak akan pernah memiliki *in-degree* 0. Hal ini secara sukses mendemonstrasikan pertahanan sistem dari kurikulum yang cacat logika.

8. Analisis Kompleksitas

Operasi	Algoritma	Kompleksitas Waktu	Penjelasan
Insert / Search (AVL)	AVL Tree	$O(\log n)$	Sifat self-balancing binary tree mengamankan tinggi tree untuk meminimalkan durasi perambatan pencarian.
Traversal Graph	DFS	$O(V + E)$	Membutuhkan kunjungan pada seluruh verteks (V) dan pinggir/edge (E) yang terhubung.

Topological Sort	Kahn / DFS Modif	$O(V + E)$	Tumbuh linear seiring penambahan ukuran jaringan prasyarat pada kurikulum.
Cycle Detection	DFS Rekursif	$O(V + E)$	Menambahkan satu memori bantu call stack array namun kompleksitas waktu tetap proporsional.

9. What-If Analysis

1. **Skala data membengkak hingga 10.000 Node:** Secara teori, memori Adjacency List dan kompleksitas memori AVL akan naik, tapi masih dalam batas aman. Waktu load data dari file CSV akan sedikit lebih lama. Namun, karena menggunakan AVL Tree untuk pencarian, performanya masih akan sangat cepat $O(\log n)$.
2. **Perubahan Konteks Bobot Edge:** Saat ini *edge* diimplementasikan tanpa bobot karena prasyarat bersifat absolut (harus diambil/tidak). Bila dikembangkan dengan bobot (misal prioritas nilai/SKS), bisa ganti menggunakan implementasi algoritma seperti Dijkstra's *shortest path* atau perhitungan lintasan kritis untuk memprediksi lulusan tercepat.
3. **Data Mengandung Inkonsistensi Duplikasi atau Kosong:** Program harus punya *Exception Handling*. Jika satu baris relasi memuat *null* atau duplikat di tahap *loading* CSV, sistem harus dengan aman melewati (skip) baris tersebut dan menampilkan log peringatan ke *console* agar sistem tidak *crash* semua.

10. Hasil & Demonstrasi

10.1. Cari matkul berdasarkan kode / prefix

```
-----
MENU UTAMA
-----

[TREE - AVL Tree]
  1. Cari matkul berdasarkan kode / prefix
  9. Tambah matkul baru
10. Hapus matkul
[GRAPH - Directed Graph]
  2. Tampilkan prasyarat langsung & tidak langsung
  3. Rekomendasi urutan pengambilan (Topological Sort)
  4. Deteksi siklus prasyarat (Cycle Detection)
  5. Telusuri rantai matkul dari suatu titik (DFS)
  7. Tampilkan seluruh graph prasyarat
[UMUM]
  6. Tampilkan semua matkul (In-Order AVL)
  8. SIMULASI: Tambah matkul baru & lihat dampaknya
  0. Keluar
-----

Pilihan Anda: 1

--- CARI MATKUL ---
Masukkan kode / prefix (contoh: IT3, IT401): IT101

[Hasil Tepat]
  IT101 - Algorithms and Programming Techniques (4 SKS)
  Prasyarat langsung: tidak ada
```

10.2. Tampilkan prasyarat langsung & tidak langsung

```
-----
MENU UTAMA
-----
[TREE - AVL Tree]
  1. Cari matkul berdasarkan kode / prefix
  9. Tambah matkul baru
10. Hapus matkul
[GRAPH - Directed Graph]
  2. Tampilkan prasyarat langsung & tidak langsung
  3. Rekomendasi urutan pengambilan (Topological Sort)
  4. Deteksi siklus prasyarat (Cycle Detection)
  5. Telusuri rantai matkul dari suatu titik (DFS)
  7. Tampilkan seluruh graph prasyarat
[UMUM]
  6. Tampilkan semua matkul (In-Order AVL)
  8. SIMULASI: Tambah matkul baru & lihat dampaknya
  0. Keluar
-----

Pilihan Anda: 2

--- TAMPILKAN PRASYARAT ---
Masukkan kode matkul (contoh: IT301): IT101

Matkul: IT101 - Algorithms and Programming Techniques (4 SKS)

Prasyarat LANGSUNG (0):
  (tidak ada - matkul ini bisa langsung diambil)

Prasyarat TIDAK LANGSUNG (0):
  (tidak ada)
```

MENU UTAMA

[TREE - AVL Tree]

1. Cari matkul berdasarkan kode / prefix
9. Tambah matkul baru
10. Hapus matkul

[GRAPH - Directed Graph]

2. Tampilkan prasyarat langsung & tidak langsung
3. Rekomendasi urutan pengambilan (Topological Sort)
4. Deteksi siklus prasyarat (Cycle Detection)
5. Telusuri rantai matkul dari suatu titik (DFS)
7. Tampilkan seluruh graph prasyarat

[UMUM]

6. Tampilkan semua matkul (In-Order AVL)
 8. SIMULASI: Tambah matkul baru & lihat dampaknya
 0. Keluar
-

Pilihan Anda: 2

--- TAMPILKAN PRASYARAT ---

Masukkan kode matkul (contoh: IT301): IT301

Matkul: IT301 - Ethical Hacking and cybersecurity testing (4 SKS)

Prasyarat LANGSUNG (4):

- IT103 (Information Technology Law and Ethics)
- IT104 (Introduction to Electrical Engineering and Intelligent Informatics)
- IT203 (Sistem Operation)
- IT204 (Data Structures and Object-Oriented Programming)

Prasyarat TIDAK LANGSUNG (2):

- IT101 (Algorithms and Programming Techniques)
- IT102 (Arsitektur and Organisation Computer)

MENU UTAMA

[TREE - AVL Tree]

1. Cari matkul berdasarkan kode / prefix
9. Tambah matkul baru
10. Hapus matkul

[GRAPH - Directed Graph]

2. Tampilkan prasyarat langsung & tidak langsung
3. Rekomendasi urutan pengambilan (Topological Sort)
4. Deteksi siklus prasyarat (Cycle Detection)
5. Telusuri rantai matkul dari suatu titik (DFS)
7. Tampilkan seluruh graph prasyarat

[UMUM]

6. Tampilkan semua matkul (In-Order AVL)
 8. SIMULASI: Tambah matkul baru & lihat dampaknya
 0. Keluar
-

Pilihan Anda: 2

--- TAMPILKAN PRASYARAT ---

Masukkan kode matkul (contoh: IT301): IT606

Matkul 'IT606' tidak ditemukan.

10.3. Rekomendasi urutan pengambilan (Topological Sort)

```
-----
MENU UTAMA
-----
[TREE - AVL Tree]
  1. Cari matkul berdasarkan kode / prefix
  9. Tambah matkul baru
10. Hapus matkul
[GRAPH - Directed Graph]
  2. Tampilkan prasyarat langsung & tidak langsung
  3. Rekomendasi urutan pengambilan (Topological Sort)
  4. Deteksi siklus prasyarat (Cycle Detection)
  5. Telusuri rantai matkul dari suatu titik (DFS)
  7. Tampilkan seluruh graph prasyarat
[UMUM]
  6. Tampilkan semua matkul (In-Order AVL)
  8. SIMULASI: Tambah matkul baru & lihat dampaknya
  0. Keluar
-----

Pilihan Anda: 3

===== TOPOLOGICAL SORT (Urutan Pengambilan MK) =====
Urutan rekomendasi pengambilan matkul:
  1. IT101
  2. IT102
  3. IT103
  4. IT104
  5. IT105
  6. IT201
  7. IT202
  8. IT203
  9. IT204
 10. IT301
 11. IT302
 12. IT303
 13. IT304
 14. IT305
 15. IT306
 16. IT401
 17. IT402
 18. IT403
 19. IT404
 20. IT405
 21. IT406
 22. IT501
 23. IT502
 24. IT503
 25. IT504
 26. IT505
 27. IT506
 28. IT601
 29. IT602
```

10.4. Deteksi siklus prasyarat (Cycle Detection)

```
MENU UTAMA
-----
[TREE - AVL Tree]
 1. Cari matkul berdasarkan kode / prefix
 9. Tambah matkul baru
10. Hapus matkul
[GRAPH - Directed Graph]
 2. Tampilkan prasyarat langsung & tidak langsung
 3. Rekomendasi urutan pengambilan (Topological Sort)
 4. Deteksi siklus prasyarat (Cycle Detection)
 5. Telusuri rantai matkul dari suatu titik (DFS)
 7. Tampilkan seluruh graph prasyarat
[UMUM]
 6. Tampilkan semua matkul (In-Order AVL)
 8. SIMULASI: Tambah matkul baru & lihat dampaknya
 0. Keluar
-----
Pilihan Anda: 4

===== CYCLE DETECTION =====
Tidak ada siklus. Kurikulum VALID.
```

```
Pilihan Anda: 4

===== CYCLE DETECTION =====
Siklus ditemukan: IT202 -> IT101 -> IT202
Kurikulum TIDAK VALID! Ada prasyarat melingkar.
-----
```

10.5. Telusuri rantai matkul dari suatu titik (DFS)

```
-----
MENU UTAMA
-----
[TREE - AVL Tree]
  1. Cari matkul berdasarkan kode / prefix
  9. Tambah matkul baru
10. Hapus matkul
[GRAPH - Directed Graph]
  2. Tampilkan prasyarat langsung & tidak langsung
  3. Rekomendasi urutan pengambilan (Topological Sort)
  4. Deteksi siklus prasyarat (Cycle Detection)
  5. Telusuri rantai matkul dari suatu titik (DFS)
  7. Tampilkan seluruh graph prasyarat
[UMUM]
  6. Tampilkan semua matkul (In-Order AVL)
  8. SIMULASI: Tambah matkul baru & lihat dampaknya
  0. Keluar
-----

Pilihan Anda: 5

--- DFS: RANTAI MATKUL ---
Masukkan kode matkul awal (contoh: IT101): IT101

===== DFS TRAVERSAL dari IT101 =====
(Menampilkan rantai matkul yang terbuka setelah IT101)
-> IT101
  -> IT202
    -> IT306
      -> IT501
      -> IT601
      -> IT602
    -> IT401
    -> IT402
    -> IT403
    -> IT503
  -> IT203
    -> IT301
    -> IT404
    -> IT302
    -> IT303
    -> IT405
    -> IT304
    -> IT406
    -> IT504
  -> IT204
  -> IT502
-----
```

10.6. Tampilkan semua matkul (In-Order AVL)

```
7. Tampilkan seluruh graph prasyarat
[UMUM]
6. Tampilkan semua matkul (In-Order AVL)
8. SIMULASI: Tambah matkul baru & lihat dampaknya
0. Keluar
-----
Pilihan Anda: 6

--- SEMUA MATA KULIAH (In-Order AVL Tree, urut kode) ---
KODE      NAMA                                     SKS    SMT TIPE      KUOTA  DEPARTEMEN
-----
IT101     Algorithms and Programming Techniques           4      1 Wajib       80      Departemen Teknologi Informasi
IT102     Arsitektur and Organitation Computer            3      1 Wajib      120      Departemen Teknologi Informasi
IT103     Information Technology Law and Ethics            2      1 Wajib       90      Departemen Teknologi Informasi
IT104     Introduction to Electrical Engineering and Intelligent Informatics  2      1 Wajib      140      Departemen Teknologi Informasi
IT105     Statistics and Probabilitys                     2      1 Wajib      110      Departemen Teknologi Informasi
IT201     Arsitektur Enterprise                           3      2 Wajib       85      Departemen Teknologi Informasi
IT202     Sistem Data Base                               4      2 Wajib       95      Departemen Teknologi Informasi
IT203     Sistem Operation                               4      2 Wajib      105      Departemen Teknologi Informasi
IT204     Data Structures and Object-Oriented Programming  3      2 Wajib      115      Departemen Teknologi Informasi
IT301     Ethical Hacking and cybersecurity testing        4      3 Wajib      125      Departemen Teknologi Informasi
IT302     Internet of Things                             3      3 Wajib      135      Departemen Teknologi Informasi
IT303     computer network security                      3      3 Wajib       88      Departemen Teknologi Informasi
IT304     Communication Data and Network Computer         4      3 Wajib       98      Departemen Teknologi Informasi
IT305     Profesional Communication                      2      3 Wajib      108      Departemen Teknologi Informasi
IT306     Web Programing                                 4      3 Wajib      118      Departemen Teknologi Informasi
IT401     Big Data and Data Lakehouse                    3      4 Wajib      128      Departemen Teknologi Informasi
IT402     Integration System                             3      4 Wajib      138      Departemen Teknologi Informasi
IT403     Artificial Inteligent and Machine Learning       4      4 Wajib       82      Departemen Teknologi Informasi
IT404     Management Accident Cybersecurity Attack        3      4 Wajib       92      Departemen Teknologi Informasi
IT405     Security Operations Center                     3      4 Wajib      102      Departemen Teknologi Informasi
IT406     Cloud Computer                                 4      4 Wajib      112      Departemen Teknologi Informasi
IT501     Web Security and Aplication                    3      5 Pilihan     122      Departemen Teknologi Informasi
IT502     Kriptografi                                    3      5 Pilihan     132      Departemen Teknologi Informasi
IT503     Mobile Programing                              3      5 Pilihan      84      Departemen Teknologi Informasi
IT504     Cloud Computing Optimization Strategy            3      5 Pilihan      94      Departemen Teknologi Informasi
IT505     Information Technology Governance               3      5 Pilihan     104      Departemen Teknologi Informasi
IT506     Smart City Technology                          3      5 Pilihan     114      Departemen Teknologi Informasi
IT702     Promt Engineering                              3      7 Pilihan      80      Departemen Teknologi Informasi
Total: 28 mata kuliah.
```

10.7. Tampilkan seluruh graph prasyarat

```
-----
MENU UTAMA
-----
[TREE - AVL Tree]
  1. Cari matkul berdasarkan kode / prefix
  9. Tambah matkul baru
 10. Hapus matkul
[GRAPH - Directed Graph]
  2. Tampilkan prasyarat langsung & tidak langsung
  3. Rekomendasi urutan pengambilan (Topological Sort)
  4. Deteksi siklus prasyarat (Cycle Detection)
  5. Telusuri rantai matkul dari suatu titik (DFS)
  7. Tampilkan seluruh graph prasyarat
[UMUM]
  6. Tampilkan semua matkul (In-Order AVL)
  8. SIMULASI: Tambah matkul baru & lihat dampaknya
  0. Keluar
-----
Pilihan Anda: 7

===== COURSE PREREQUISITE GRAPH =====
Format: [Matkul] -> [Matkul yang membutuhkannya sebagai prasyarat]
IT101 -> IT202, IT203, IT204, IT302, IT306, IT401, IT502
IT102 -> IT201, IT203, IT304
IT103 -> IT301, IT303
IT104 -> IT301, IT403
IT105 -> IT401
IT201 -> IT402
IT202 -> IT306, IT401, IT402, IT403, IT503
IT203 -> IT301, IT302, IT303, IT304, IT406
IT204 -> IT301, IT306, IT403, IT503
IT301 -> IT404, IT501
IT302 -> (tidak menjadi prasyarat matkul lain)
IT303 -> IT404, IT405
IT304 -> IT405, IT406
IT305 -> (tidak menjadi prasyarat matkul lain)
IT306 -> IT501, IT601, IT602
IT401 -> (tidak menjadi prasyarat matkul lain)
IT402 -> (tidak menjadi prasyarat matkul lain)
IT403 -> (tidak menjadi prasyarat matkul lain)
IT404 -> (tidak menjadi prasyarat matkul lain)
IT405 -> (tidak menjadi prasyarat matkul lain)
IT406 -> IT504, IT602
IT501 -> (tidak menjadi prasyarat matkul lain)
IT502 -> (tidak menjadi prasyarat matkul lain)
IT503 -> (tidak menjadi prasyarat matkul lain)
IT504 -> (tidak menjadi prasyarat matkul lain)
IT505 -> (tidak menjadi prasyarat matkul lain)
IT506 -> (tidak menjadi prasyarat matkul lain)
IT601 -> (tidak menjadi prasyarat matkul lain)
IT602 -> (tidak menjadi prasyarat matkul lain)
```

10.8. SIMULASI: Tambah matkul baru & lihat dampaknya

```
-----
MENU UTAMA
-----
[TREE - AVL Tree]
  1. Cari matkul berdasarkan kode / prefix
  9. Tambah matkul baru
  10. Hapus matkul
[GRAPH - Directed Graph]
  2. Tampilkan prasyarat langsung & tidak langsung
  3. Rekomendasi urutan pengambilan (Topological Sort)
  4. Deteksi siklus prasyarat (Cycle Detection)
  5. Telusuri rantai matkul dari suatu titik (DFS)
  7. Tampilkan seluruh graph prasyarat
[UMUM]
  6. Tampilkan semua matkul (In-Order AVL)
  8. SIMULASI: Tambah matkul baru & lihat dampaknya
  0. Keluar
-----
Pilihan Anda: 8

===== SIMULASI: PENAMBAHAN MATKUL BARU =====
Fitur ini mensimulasikan penambahan matkul baru ke kurikulum
dan menganalisis efeknya terhadap urutan pengambilan matkul.

Kode matkul baru (contoh: IT507): IT809
Nama matkul: Belajar Prompting
SKS: 4
Semester: 8
Tipe (Wajib/Pilihan): Pilihan

Masukkan prasyarat matkul ini (pisahkan dengan koma, kosongkan jika tidak ada):
Prasyarat:

===== SIMULASI: PENAMBAHAN MATKUL BARU =====
Matkul baru: IT809
Prasyarat: (tidak ada)

--- Analisis Dampak ---
Jumlah matkul sebelum: 29
Jumlah matkul sesudah: 30
Status: Kurikulum tetap VALID (tidak ada siklus).
Posisi IT809 dalam urutan pengambilan: ke-30

Urutan matkul sesudah penambahan IT809:
  1. IT101
  2. IT102
  3. IT103
  4. IT104
  5. IT105
  6. IT201
  7. IT202
  8. IT203
  9. IT204
 10. IT301
 11. IT302
 12. IT303
 13. IT304
 14. IT305
 15. IT306
 16. IT401
 17. IT402
 18. IT403
 19. IT404
 20. IT405
 21. IT406
 22. IT501
 23. IT502
 24. IT503
 25. IT504
 26. IT505
 27. IT506
 28. IT601
 29. IT602
 30. IT809 <-- BARU

[INFO] Matkul IT809 berhasil ditambahkan ke sistem.
Gunakan menu 3 untuk melihat urutan lengkap terbaru.
```

10.9. Tambah matkul baru

MENU UTAMA

[TREE - AVL Tree]

1. Cari matkul berdasarkan kode / prefix

9. Tambah matkul baru

10. Hapus matkul

[GRAPH - Directed Graph]

2. Tampilkan prasyarat langsung & tidak langsung

3. Rekomendasi urutan pengambilan (Topological Sort)

4. Deteksi siklus prasyarat (Cycle Detection)

5. Telusuri rantai matkul dari suatu titik (DFS)

7. Tampilkan seluruh graph prasyarat

[UMUM]

6. Tampilkan semua matkul (In-Order AVL)

8. SIMULASI: Tambah matkul baru & lihat dampaknya

0. Keluar

Pilihan Anda: 9

--- TAMBAH MATA KULIAH ---

Kode (contoh: IT601): IT702

Nama: Promt Engineering

SKS: 3

Semester: 7

Tipe (Wajib/Pilihan): Pilihan

Kuota: IT403

[!] Input tidak valid! Harap masukkan angka bulat yang benar.

Kuota: 80

Departemen Penyelenggara: Departemen Teknologi Informasi

Prasyarat (pisah koma, kosong = tidak ada): IT403

[OK] Matkul 'IT702 - Promt Engineering' berhasil ditambahkan.

MENU UTAMA

[TREE - AVL Tree]

1. Cari matkul berdasarkan kode / prefix

9. Tambah matkul baru

10. Hapus matkul

[GRAPH - Directed Graph]

2. Tampilkan prasyarat langsung & tidak langsung

3. Rekomendasi urutan pengambilan (Topological Sort)

4. Deteksi siklus prasyarat (Cycle Detection)

5. Telusuri rantai matkul dari suatu titik (DFS)

7. Tampilkan seluruh graph prasyarat

[UMUM]

6. Tampilkan semua matkul (In-Order AVL)

8. SIMULASI: Tambah matkul baru & lihat dampaknya

0. Keluar

Pilihan Anda: 9

--- TAMBAH MATA KULIAH ---

Kode (contoh: IT601): IT809

[!] Matkul 'IT809' sudah ada.

10.10. Hapus matkul

```
-----
MENU UTAMA
-----
[TREE - AVL Tree]
  1. Cari matkul berdasarkan kode / prefix
  9. Tambah matkul baru
10. Hapus matkul
[GRAPH - Directed Graph]
  2. Tampilkan prasyarat langsung & tidak langsung
  3. Rekomendasi urutan pengambilan (Topological Sort)
  4. Deteksi siklus prasyarat (Cycle Detection)
  5. Telusuri rantai matkul dari suatu titik (DFS)
  7. Tampilkan seluruh graph prasyarat
[UMUM]
  6. Tampilkan semua matkul (In-Order AVL)
  8. SIMULASI: Tambah matkul baru & lihat dampaknya
  0. Keluar
-----
Pilihan Anda: 10

--- HAPUS MATA KULIAH ---
Kode matkul yang akan dihapus: IT801
Matkul ditemukan: IT801 - Jailbreaking Fable (2 SKS)
Yakin ingin menghapus? (y/n): y
[OK] Matkul 'IT801' berhasil dihapus dari AVL Tree dan Graph.
-----
```

```
-----
MENU UTAMA
-----
[TREE - AVL Tree]
  1. Cari matkul berdasarkan kode / prefix
  9. Tambah matkul baru
10. Hapus matkul
[GRAPH - Directed Graph]
  2. Tampilkan prasyarat langsung & tidak langsung
  3. Rekomendasi urutan pengambilan (Topological Sort)
  4. Deteksi siklus prasyarat (Cycle Detection)
  5. Telusuri rantai matkul dari suatu titik (DFS)
  7. Tampilkan seluruh graph prasyarat
[UMUM]
  6. Tampilkan semua matkul (In-Order AVL)
  8. SIMULASI: Tambah matkul baru & lihat dampaknya
  0. Keluar
-----
Pilihan Anda: 1

--- CARI MATKUL ---
Masukkan kode / prefix (contoh: IT3, IT401): IT801
Tidak ada matkul dengan prefix 'IT801'.
```

MENU UTAMA

[TREE - AVL Tree]

1. Cari matkul berdasarkan kode / prefix
9. Tambah matkul baru
10. Hapus matkul

[GRAPH - Directed Graph]

2. Tampilkan prasyarat langsung & tidak langsung
3. Rekomendasi urutan pengambilan (Topological Sort)
4. Deteksi siklus prasyarat (Cycle Detection)
5. Telusuri rantai matkul dari suatu titik (DFS)
7. Tampilkan seluruh graph prasyarat

[UMUM]

6. Tampilkan semua matkul (In-Order AVL)
8. SIMULASI: Tambah matkul baru & lihat dampaknya
0. Keluar

Pilihan Anda: 10

--- HAPUS MATA KULIAH ---

Kode matkul yang akan dihapus: IT801

[!] Matkul 'IT801' tidak ditemukan.